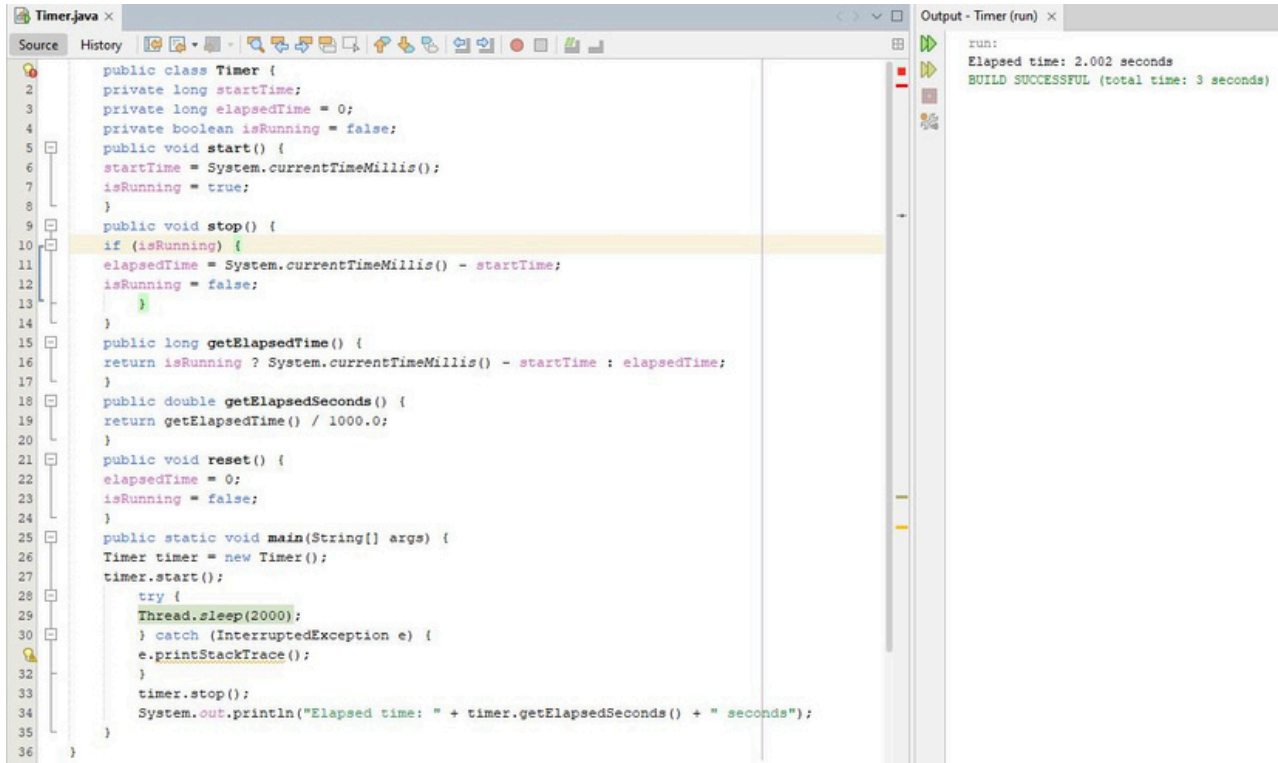


Week 3 Test Results

Timer.java



The screenshot shows an IDE window titled "Timer.java" with a source code editor and an output console. The code defines a `Timer` class with methods for starting, stopping, and getting elapsed time. The `main` method creates a `Timer` object, starts it, sleeps for 2000 milliseconds, and then stops it, printing the elapsed time in seconds.

```
1 public class Timer {
2     private long startTime;
3     private long elapsedTime = 0;
4     private boolean isRunning = false;
5     public void start() {
6         startTime = System.currentTimeMillis();
7         isRunning = true;
8     }
9     public void stop() {
10        if (isRunning) {
11            elapsedTime = System.currentTimeMillis() - startTime;
12            isRunning = false;
13        }
14    }
15    public long getElapsedTime() {
16        return isRunning ? System.currentTimeMillis() - startTime : elapsedTime;
17    }
18    public double getElapsedSeconds() {
19        return getElapsedTime() / 1000.0;
20    }
21    public void reset() {
22        elapsedTime = 0;
23        isRunning = false;
24    }
25    public static void main(String[] args) {
26        Timer timer = new Timer();
27        timer.start();
28        try {
29            Thread.sleep(2000);
30        } catch (InterruptedException e) {
31            e.printStackTrace();
32        }
33        timer.stop();
34        System.out.println("Elapsed time: " + timer.getElapsedSeconds() + " seconds");
35    }
36 }
```

The output console shows the following text:

```
run:
Elapsed time: 2.002 seconds
BUILD SUCCESSFUL (total time: 3 seconds)
```

WordMatcher.java

The screenshot displays an IDE with two panes. The left pane shows the source code for `WordMatcher.java`, and the right pane shows the output of running the program.

Source Code (WordMatcher.java):

```
1 public class WordMatcher {
2     private String correctWord;
3     private String userInput;
4     public WordMatcher(String correctWord, String userInput) {
5         this.correctWord = correctWord;
6         this.userInput = userInput;
7     }
8     public boolean isExactMatch() {
9         return correctWord.equals(userInput);
10    }
11    public int getCorrectChars() {
12        int correctCount = 0;
13        int minLength = Math.min(correctWord.length(), userInput.length());
14        for (int i = 0; i < minLength; i++) {
15            if (correctWord.charAt(i) == userInput.charAt(i)) {
16                correctCount++;
17            }
18        }
19        return correctCount;
20    }
21    public int getIncorrectChars() {
22        return userInput.length() - getCorrectChars();
23    }
24    public static void main(String[] args) {
25        System.out.println("=== WordMatcher Test ===\n");
26        WordMatcher test1 = new WordMatcher("hello", "hello");
27        System.out.println("Test 1: Exact Match");
28        System.out.println("Correct word: hello");
29        System.out.println("User input: hello");
30        System.out.println("Is exact match: " + test1.isExactMatch());
31        System.out.println("Correct chars: " + test1.getCorrectChars());
32        System.out.println("Incorrect chars: " + test1.getIncorrectChars());
33        System.out.println();
34
35        WordMatcher test2 = new WordMatcher("hello", "hallo");
36        System.out.println("Test 2: Partial Match");
37        System.out.println("Correct word: hello");
38        System.out.println("User input: hallo");
39        System.out.println("Is exact match: " + test2.isExactMatch());
40        System.out.println("Correct chars: " + test2.getCorrectChars());
41        System.out.println("Incorrect chars: " + test2.getIncorrectChars());
42        System.out.println();
43
44        WordMatcher test3 = new WordMatcher("computer", "compter");
45        System.out.println("Test 3: Multiple Errors");
46        System.out.println("Correct word: computer");
47        System.out.println("User input: compter");
48        System.out.println("Is exact match: " + test3.isExactMatch());
49        System.out.println("Correct chars: " + test3.getCorrectChars());
50        System.out.println("Incorrect chars: " + test3.getIncorrectChars());
51    }
52 }
```

Output (WordMatcher (run)):

```
run:
=== WordMatcher Test ===

Test 1: Exact Match
Correct word: hello
User input: hello
Is exact match: true
Correct chars: 5
Incorrect chars: 0

Test 2: Partial Match
Correct word: hello
User input: hallo
Is exact match: false
Correct chars: 4
Incorrect chars: 1

Test 3: Multiple Errors
Correct word: computer
User input: compter
Is exact match: false
Correct chars: 4
Incorrect chars: 3
BUILD SUCCESSFUL (total time: 0 seconds)
```

AccuracyCalculator.java

```
Timer.java x WordMatcher.java x AccuracyCalculator.java x
Source History
1 public class AccuracyCalculator {
2     private int correctChars;
3     private int totalChars;
4     private double timeInSeconds;
5     public AccuracyCalculator(int correctChars, int totalChars, double timeInSeconds) {
6         this.correctChars = correctChars;
7         this.totalChars = totalChars;
8         this.timeInSeconds = timeInSeconds;
9     }
10    public double calculateWPM() {
11        double timeInMinutes = timeInSeconds / 60.0;
12        double wordsTyped = totalChars / 5.0;
13        return wordsTyped / timeInMinutes;
14    }
15    public double calculateAccuracyPercent() {
16        if (totalChars == 0) return 0;
17        return (correctChars / (double) totalChars) * 100;
18    }
19    public void printResults() {
20        System.out.println("=== Typing Test Results ===");
21        System.out.printf("WPM: %.2f\n", calculateWPM());
22        System.out.printf("Accuracy: %.2f%%\n", calculateAccuracyPercent());
23    }
24    public static void main(String[] args) {
25        System.out.println("=== AccuracyCalculator Test ===\n");
26        System.out.println("Test 1: Perfect Typing");
27        AccuracyCalculator test1 = new AccuracyCalculator(50, 50, 30);
28        System.out.println("Correct chars: 50, Total chars: 50, Time: 30 seconds");
29        test1.printResults();
30        System.out.println();
31        System.out.println("Test 2: Good Typing (90% Accuracy)");
32        AccuracyCalculator test2 = new AccuracyCalculator(45, 50, 30);
33        System.out.println("Correct chars: 45, Total chars: 50, Time: 30 seconds");
34        test2.printResults();
35        System.out.println();
36        System.out.println("Test 3: Fast Typing");
37        AccuracyCalculator test3 = new AccuracyCalculator(80, 85, 15);
38        System.out.println("Correct chars: 80, Total chars: 85, Time: 15 seconds");
39        test3.printResults();
40        System.out.println();
41        System.out.println("Test 4: Slow Typing");
42        AccuracyCalculator test4 = new AccuracyCalculator(30, 35, 60);
43        System.out.println("Correct chars: 30, Total chars: 35, Time: 60 seconds");
44        test4.printResults();
45    }
46 }
```

```
run:
=== AccuracyCalculator Test ===

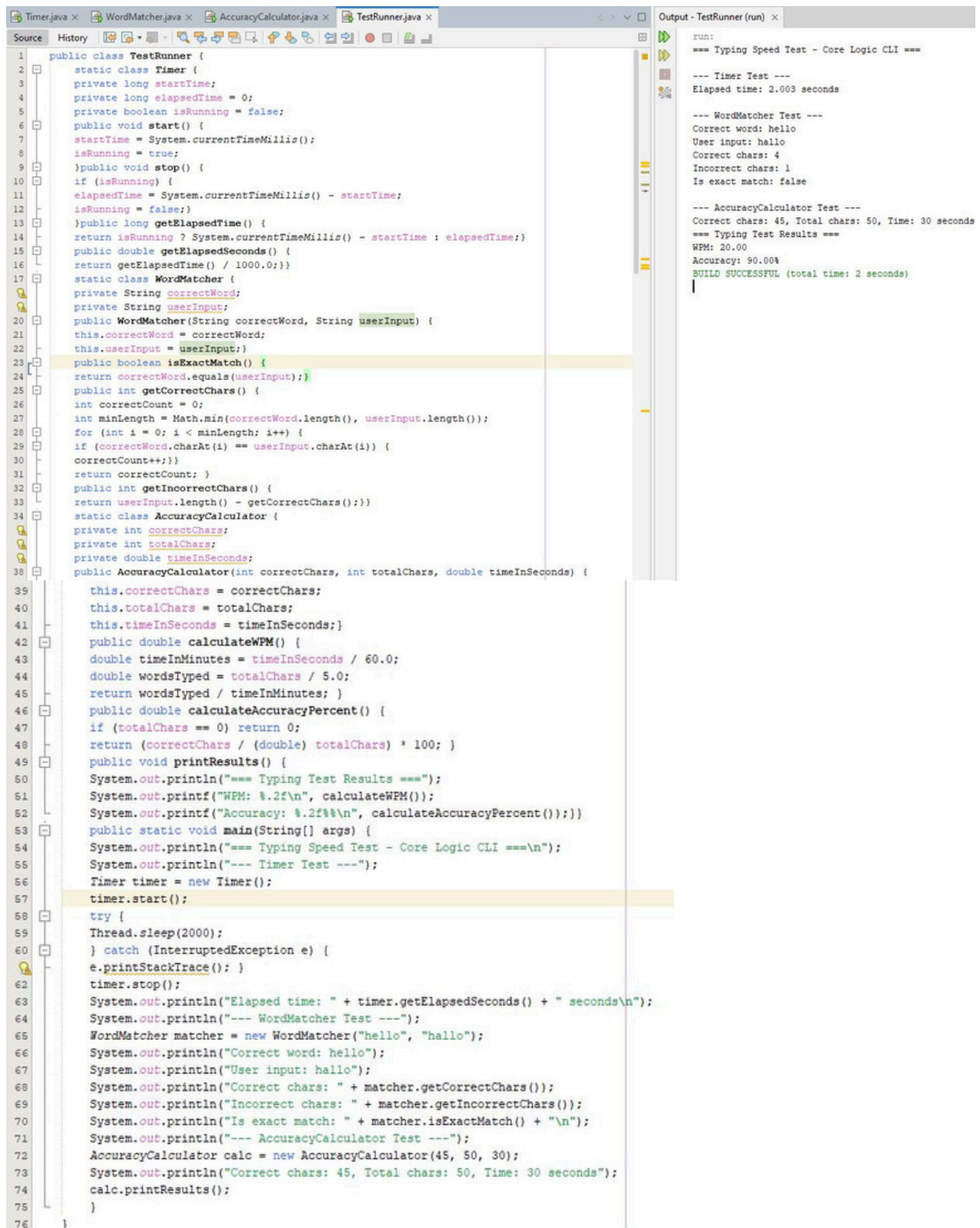
Test 1: Perfect Typing
Correct chars: 50, Total chars: 50, Time: 30 seconds
=== Typing Test Results ===
WPM: 20.00
Accuracy: 100.00%

Test 2: Good Typing (90% Accuracy)
Correct chars: 45, Total chars: 50, Time: 30 seconds
=== Typing Test Results ===
WPM: 20.00
Accuracy: 90.00%

Test 3: Fast Typing
Correct chars: 80, Total chars: 85, Time: 15 seconds
=== Typing Test Results ===
WPM: 68.00
Accuracy: 94.12%

Test 4: Slow Typing
Correct chars: 30, Total chars: 35, Time: 60 seconds
=== Typing Test Results ===
WPM: 7.00
Accuracy: 85.71%
BUILD SUCCESSFUL (total time: 0 seconds)
```

TestRunner.java CLI Application



The screenshot displays an IDE with the `TestRunner.java` file open in the source editor. The code defines a `TestRunner` class that manages a `Timer`, a `WordMatcher`, and an `AccuracyCalculator`. The `main` method executes a series of tests, including a typing speed test, a word matching test, and an accuracy calculation test. The output window on the right shows the results of these tests, including elapsed time, correct/incorrect characters, and accuracy percentages.

```
1 public class TestRunner {
2     static class Timer {
3         private long startTime;
4         private long elapsedTime = 0;
5         private boolean isRunning = false;
6         public void start() {
7             startTime = System.currentTimeMillis();
8             isRunning = true;
9         }
10        public void stop() {
11            if (isRunning) {
12                elapsedTime = System.currentTimeMillis() - startTime;
13                isRunning = false;
14            }
15        }
16        public long getElapsedTime() {
17            return isRunning ? System.currentTimeMillis() - startTime : elapsedTime;
18        }
19        public double getElapsedSeconds() {
20            return getElapsedTime() / 1000.0;
21        }
22        static class WordMatcher {
23            private String correctWord;
24            private String userInput;
25            public WordMatcher(String correctWord, String userInput) {
26                this.correctWord = correctWord;
27                this.userInput = userInput;
28            }
29            public boolean isExactMatch() {
30                return correctWord.equals(userInput);
31            }
32            public int getCorrectChars() {
33                int correctCount = 0;
34                int minLength = Math.min(correctWord.length(), userInput.length());
35                for (int i = 0; i < minLength; i++) {
36                    if (correctWord.charAt(i) == userInput.charAt(i)) {
37                        correctCount++;
38                    }
39                }
40                return correctCount;
41            }
42            public int getIncorrectChars() {
43                return userInput.length() - getCorrectChars();
44            }
45            static class AccuracyCalculator {
46                private int correctChars;
47                private int totalChars;
48                private double timeInSeconds;
49                public AccuracyCalculator(int correctChars, int totalChars, double timeInSeconds) {
50                    this.correctChars = correctChars;
51                    this.totalChars = totalChars;
52                    this.timeInSeconds = timeInSeconds;
53                }
54                public double calculateWPM() {
55                    double timeInMinutes = timeInSeconds / 60.0;
56                    double wordsTyped = totalChars / 5.0;
57                    return wordsTyped / timeInMinutes;
58                }
59                public double calculateAccuracyPercent() {
60                    if (totalChars == 0) return 0;
61                    return (correctChars / (double) totalChars) * 100;
62                }
63                public void printResults() {
64                    System.out.println("=== Typing Test Results ===");
65                    System.out.printf("WPM: %.2f\n", calculateWPM());
66                    System.out.printf("Accuracy: %.2f%%\n", calculateAccuracyPercent());
67                }
68                public static void main(String[] args) {
69                    System.out.println("=== Typing Speed Test - Core Logic CLI ===\n");
70                    System.out.println("--- Timer Test ---");
71                    Timer timer = new Timer();
72                    timer.start();
73                    try {
74                        Thread.sleep(2000);
75                    } catch (InterruptedException e) {
76                        e.printStackTrace();
77                    }
78                    timer.stop();
79                    System.out.println("Elapsed time: " + timer.getElapsedSeconds() + " seconds\n");
80                    System.out.println("--- WordMatcher Test ---");
81                    WordMatcher matcher = new WordMatcher("hello", "hallo");
82                    System.out.println("Correct word: hello");
83                    System.out.println("User input: hallo");
84                    System.out.println("Correct chars: " + matcher.getCorrectChars());
85                    System.out.println("Incorrect chars: " + matcher.getIncorrectChars());
86                    System.out.println("Is exact match: " + matcher.isExactMatch() + "\n");
87                    System.out.println("--- AccuracyCalculator Test ---");
88                    AccuracyCalculator calc = new AccuracyCalculator(45, 50, 30);
89                    System.out.println("Correct chars: 45, Total chars: 50, Time: 30 seconds");
90                    calc.printResults();
91                }
92            }
93        }
94    }
95 }
```

Output - TestRunner (run) x

```
run:
=== Typing Speed Test - Core Logic CLI ===

--- Timer Test ---
Elapsed time: 2.003 seconds

--- WordMatcher Test ---
Correct word: hello
User input: hallo
Correct chars: 4
Incorrect chars: 1
Is exact match: false

--- AccuracyCalculator Test ---
Correct chars: 45, Total chars: 50, Time: 30 seconds
=== Typing Test Results ===
WPM: 20.00
Accuracy: 90.00%
BUILD SUCCESSFUL (total time: 2 seconds)
```